KYC API Documentation

Underwriting Tool Backend - KYC router and controller contract

Generated from repository code on 2026-06-19.

Scope

This document covers the FastAPI routes registered in routes/kyc_router.py under /v1/kyc and the controller implementation in controller/kyc_controller/aadhar_kyc_controller.py.

Area	Current implementation
Router	routes/kyc_router.py exposes nine KYC routes with prefix /v1/kyc.
Controller	controller/kyc_controller/aadhar_kyc_controller.py validates input, calls ScoreMe KYC APIs, persists data in MongoDB, and returns JSON responses.
Auth	Global authorization middleware protects every KYC route with the access_token cookie.
Upstream provider	ScoreMe KYC sandbox endpoints are configured in config/config.py.
Persistence	MongoDB collections used: aadhaar_otp_manager, aadhaar_details, digilocker_session_manager, user_kyc_documents.

Base URL And Authentication

```
Base path: {API_BASE_URL}/v1/kyc
Auth cookie: access_token
Frontend fetch option: credentials: "include"
```

All KYC routes are protected because they are not listed as public routes in middleware/authorization_middleware.py.

```
await fetch(`${API_BASE_URL}/v1/kyc/aadhaar/details`, {
  method: "GET",
  credentials: "include"
});
```

Auth failures return top-level JSON instead of FastAPI's detail wrapper:

Case	HTTP	Body
Missing cookie	401	{ "message": "Unauthorized Access" }
Invalid token	401	{ "message": "Invalid Token" }

Response Conventions

Case	Shape
Successful JSONResponse	Usually top-level { "message": string, "data": any, "responseCode"?: string }.
HTTPException	FastAPI wraps controller detail as { "detail": { "message": string, "data"?: any, "responseCode"?: string } }.
Auth middleware failures	Top-level { "message": string }.
DigiLocker status values	Controller uses KYC_FLOW_STATUS enum values such as CONSENT_PENDING, IN_PROGRESS, CONSENT_APPROVED, CONSENT_REJECTED, TIMEOUT, EXPIRED, and ERROR.

Recommended frontend normalizer:

```
async function parseApiResponse(response: Response) {
  const body = await response.json().catch(() => ({}));
  const payload = body.detail || body;

  return {
    ok: response.ok,
    status: response.status,
    message: payload.message || "Request failed",
    responseCode: payload.responseCode || null,
    data: payload.data ?? null,
    raw: body
  };
```

```
}
```

Recommended KYC Flow

Step	Endpoint	Frontend action
1	POST /aadhaar/generate-otp	Collect a 12 digit Aadhaar number and request an OTP.
2	POST /aadhaar/validate-otp	Submit OTP with Aadhaar number and reference_id returned by step 1.
3	GET /aadhaar/details	Fetch stored Aadhaar details for the logged-in user.
4	GET /digilocker/generate-url	Generate a DigiLocker consent URL and redirect/open it for the user.
5	POST /digilocker/session-status	Check consent status with kyc_flow_id.
6	PATCH /digilocker/session-status	Optionally update the locally stored session status when the frontend has a status transition to persist.
7	POST /digilocker/list-documents	Fetch available documents after successful consent. Current implementation requires session_id, which is not returned by step 4.
8	POST /digilocker/document-url	Request download URL for a selected document. Current implementation requires session_id.
9	GET /digilocker/document-precheck	Reuse already fetched KYC documents for returning users.

Endpoint Reference

1. Generate Aadhaar OTP

POST /v1/kyc/aadhaar/generate-otp

Validates Aadhaar number, calls ScoreMe aadhaarOtp, and stores OTP session metadata in aadhaar_otp_manager.

Request:

```
{
  "aadhaar_number": "123456789012"
}
```

Success response:

```
{
  "message": "OTP successfully sent to mobile number",
  "data": {
    "aadhaar_number": "123456789012",
    "reference_id": "<scoreme-reference-id>"
  }
}
```

Condition	HTTP	Body
Empty body	400	{ "detail": { "message": "Body cannot be empty" } }
Empty Aadhaar number	400	{ "detail": { "message": "Aadhaar number cannot be empty" } }
Not exactly 12 digits	400	{ "detail": { "message": "Invalid aadhaar number" } }
Invalid JSON	400	{ "detail": { "message": "Invalid json" } }
ScoreMe mapped error	400, 403, 404, 429, 500, 503, or 504	{ "detail": { "message": "...", "responseCode": "...", "data": null } }
Upstream network or HTTP error	500	{ "detail": { "message": "Internal server error" } }

Frontend notes:

- Store reference_id until OTP validation completes.
- The controller directly reads input["aadhaar_number"]; send this key explicitly to avoid a generic 500 for missing key input.

2. Validate Aadhaar OTP

POST /v1/kyc/aadhaar/validate-otp

Submits Aadhaar and OTP to ScoreMe aadhaarDetail, stores Aadhaar details in aadhaar_details, and marks the OTP session complete.

Request:

```
{
  "aadhaar_number": "123456789012",
  "otp": "123456",
  "reference_id": "<reference-id-from-generate-otp>"
}
```

Success response:

```
{
  "message": "Otp validation success",
  "data": {
    "...": "ScoreMe Aadhaar data"
  }
}
```

Condition	HTTP	Body
Empty body	400	{ "detail": { "message": "Body cannot be empty" } }
Missing aadhaar_number, otp, or reference_id	400	{ "detail": { "message": "Aadhaar number and Otp and referenceId is required" } }
Mapped ScoreMe OTP error	400, 408, 429, 500, or 503	{ "detail": { "message": "...", "responseCode": "...", "data": {} } }
MongoDB error	500	{ "detail": { "message": "Internal server error" } }
Upstream network or HTTP error	500	{ "detail": { "message": "Internal server error" } }

Frontend notes:

- reference_id is used only for the local OTP session update and is not sent to ScoreMe.
- ScoreMe response code ETP011 increments otp_attempts in aadhaar_otp_manager.
- If ScoreMe returns an unmapped non-success responseCode, the controller currently has no explicit return path.

3. Fetch Aadhaar Details

GET /v1/kyc/aadhaar/details

Returns stored Aadhaar details for the authenticated user.

Request: no body.

Success response:

```
{
  "message": "aadhaar details fetched successfully",
  "data": {
    "...": "stored Aadhaar data"
  },
  "responseCode": "SYS_OK"
}
```

Condition	HTTP	Body
No Aadhaar details for user	404	{ "detail": { "message": "aadhaar details not found for this user", "data": null, "responseCode": "SYS_NOT_FOUND" } }
MongoDB error	500	{ "detail": { "message": "Internal server error" } }

Frontend note: the query excludes data.photoBase64 and data.xmlBase64.

4. Generate DigiLocker URL

GET /v1/kyc/digilocker/generate-url

Calls ScoreMe initiateDigiLocker, stores session metadata in digilocker_session_manager, and returns a local kyc_flow_id plus DigiLocker consent URL.

Request: no body.

Success response:

```
{
  "message": "Digilocker link generated successfully",
  "data": {
    "kyc_flow_id": "<local-kyc-flow-id>",
    "digilocker_url": "https://..."
  },
  "responseCode": "SYS_OKAY"
}
```

Condition	HTTP	Body
ScoreMe mapped error	400, 500, 503, or 504	{ "detail": { "message": "...", "responseCode": "...", "data": {} } }
Upstream network or HTTP error	500	{ "detail": { "message": "Internal server error", "data": null, "responseCode": "SYS_INT_ERR" } }
MongoDB error	500	{ "detail": { "message": "Internal server error", "data": null, "responseCode": "SYS_INT_ERR" } }
Unexpected non-success response	500	{ "detail": { "message": "Internal server error", "data": null, "responseCode": "SYS_INT_ERR" } }

Frontend notes:

- Use data.kyc_flow_id for POST /digilocker/session-status.
- The API does not return session_id, although later document APIs require it.
- Open data.digilocker_url in a browser tab or redirect flow for user consent.

5. Update DigiLocker Session Status

PATCH /v1/kyc/digilocker/session-status

Updates the locally stored DigiLocker session status in digilocker_session_manager.

Request:

```
{
  "session_id": "<session-id>",
  "session_status": "COMPLETED"
}
```

Success response:

```
{
  "message": "Session status updated successfully",
  "data": {
    "session_id": "<session-id>",
    "session_status": "<previous-status>"
  },
  "responseCode": "SYS_OKAY"
}
```

Condition	HTTP	Body
Empty body	400	{ "detail": { "message": "Request body cannot be empty" } }
Missing session_id or session_status	400	{ "detail": { "message": "both session_id and session_status are required" } }
Unknown session_id	404	{ "detail": { "message": "Invalid session_id" } }
Invalid JSON	400	{ "detail": { "message": "Invalid JSON", "data": null, "responseCode": "SYS_INPUT_ERROR" } }

Frontend note: the returned data is the session document fetched before update, so the echoed session_status can be the previous value.

6. Check DigiLocker Session Status

POST /v1/kyc/digilocker/session-status

Validates a local DigiLocker session by kyc_flow_id, checks local expiry, calls ScoreMe documentConsentStatus using the stored session_id, and updates local session_status.

Request:

```
{
  "kyc_flow_id": "<local-kyc-flow-id>"
}
```

Success response:

```
{
  "message": "Kyc flow status fetched successfully",
  "data": {
    "kyc_flow_id": "<local-kyc-flow-id>",
    "session_status": "CONSENT_APPROVED"
  },
  "responseCode": "SYS_OK"
}
```

Condition	HTTP	Body
Empty body	400	{ "detail": { "message": "Request body cannot be empty" } }
Missing kyc_flow_id	400	{ "detail": { "message": "kyc_flow_id is required" } }
Unknown kyc_flow_id for user	404	{ "detail": { "message": "Invalid kyc_flow_id", "data": null, "responseCode": "SYS_INPUT_ERR" } }
Local session expired	400	{ "detail": { "message": "Session expired", "data": { "kyc_flow_id": "...", "session_status": "EXPIRED" }, "responseCode": "SYS_INPUT_ERR" } }
ScoreMe timeout	504	{ "detail": { "message": "Gateway timeout error", "data": null, "responseCode": "SYS_INT_ERR" } }
Invalid JSON	400	{ "detail": { "message": "Invalid JSON", "data": null, "responseCode": "SYS_INPUT_ERROR" } }
Other exception	500	{ "detail": { "message": "Internal server error", "data": null, "responseCode": "SYS_INT_ERR" } }

ScoreMe status mapping:

ScoreMe responseCode	Stored session_status
SRC001	CONSENT_APPROVED
RNP020	IN_PROGRESS
RNP030	CONSENT_REJECTED
ERT788	TIMEOUT
Any other code	ERROR

Frontend notes:

- Poll with kyc_flow_id, not session_id.
- The current implementation places enum objects directly in some JSONResponse bodies; if not serialized by the framework version, this can cause a runtime serialization error.

7. List DigiLocker Documents

POST /v1/kyc/digilocker/list-documents

Validates a local session by session_id, calls ScoreMe documentList, and stores returned documents in user_kyc_documents.

Request:

```
{
  "session_id": "<session-id>"
}
```

Success response:

```
{
```

```
  "message": "Session status updated successfully",
  "data": {
    "user_id": "...",
    "document_list": [
      {
        "documentType": "...",
        "documentFormat": "...",
        "documentUri": "..."
      }
    ],
    "session_id": "..."
  },
  "responseCode": "SYS_OKAY"
}
```

Condition	HTTP	Body
Empty body	400	{ "detail": { "message": "Request body cannot be empty" } }
Missing session_id	400	{ "detail": { "message": "session_id is required" } }
Unknown session_id	404	{ "detail": { "message": "Invalid session_id" } }
Mapped ScoreMe document error	400, 404, 408, 500, or 503	{ "detail": { "message": "...", "responseCode": "...", "data": {} } }
Invalid JSON	400	{ "detail": { "message": "Invalid JSON", "data": null, "responseCode": "SYS_INPUT_ERROR" } }
Upstream network or HTTP error	500	{ "detail": { "message": "Internal server error" } }

Frontend notes:

- session_id is required but not returned by GET /digilocker/generate-url.
- Use documentType, documentFormat, and documentUri from the response when requesting a document URL.
- The duplicate raise_digilocker_document_list_exception function means runtime behavior uses the get-document error map.

8. Fetch DigiLocker Document URL

POST /v1/kyc/digilocker/document-url

Requests a downloadable document URL from ScoreMe and updates the matching document in user_kyc_documents.

Request:

```
{
  "session_id": "<session-id>",
  "document_format": "PDF",
  "document_uri": "<document-uri-from-list>",
  "document_type": "<documentType-from-list>"
}
```

Success response:

```
{
  "message": "Session status updated successfully",
  "data": {
    "documentUrl": "https://..."
  },
  "responseCode": "SYS_OKAY"
}
```

Condition	HTTP	Body
Empty body	400	{ "detail": { "message": "Request body cannot be empty" } }
Missing required field	400	{ "detail": { "message": "Session_id,document_format,document_uri,document_type are required" } }
No stored documents for session_id	404	{ "detail": { "message": "Invalid session_id" } }
Mapped ScoreMe document error	400, 404, 408, 500, or 503	{ "detail": { "message": "...", "responseCode": "...", "data": {} } }
Invalid JSON	400	{ "detail": { "message": "Invalid JSON", "data": null, "responseCode": "SYS_INPUT_ERROR" } }
Upstream network or HTTP error	500	{ "detail": { "message": "Internal server error" } }

Frontend notes:

- document_type is not sent to ScoreMe; it is used to update the matching local document_list item.
- If document_type does not match a stored documentType, the API can still return 200 while no local document entry is updated.

9. DigiLocker Document Precheck

GET /v1/kyc/digilocker/document-precheck

Returns already stored KYC DigiLocker documents for the authenticated user.

Request: no body.

Success response:

```
{
  "message": "Kyc documents found",
  "data": {
    "user_id": "...",
    "document_list": [
      {
        "documentType": "...",
        "documentUrl": "https://..."
      }
    ],
    "session_id": "..."
  },
  "responseCode": "SYS_OKAY"
}
```

Condition	HTTP	Body
No stored documents for user	404	{ "detail": { "message": "No Kyc Documents found for this user", "data": {}, "responseCode": "SYS_NOT_FOUND" } }

Frontend note: the controller removes document_url_fetched_at from each document before responding.

ScoreMe Error Codes Exposed To Frontend

Aadhaar OTP Generation

Code	HTTP	Message
EBF017	400	Blank Input Field.
EIP018	400	Incorrect Input.
EPI022	400	Payload is Incorrect.
EIS042	503	Information Source is Not Working.
EUP007	500	Unable To Process. Please Reach Out To Support.
EAN1229	400	Aadhaar number does not have a mobile number registered with it.
EAN1391	403	Aadhaar locked by the Aadhaar number holder.
EAE168	404	Aadhaar does not exist.
ERT788	504	Request Timed Out.
EAS517	429	OTP already sent. Please try after 120 Sec.
EML1916	429	You've reached the maximum number of attempts.

Aadhaar OTP Validation

Code	HTTP	Message
EBF017	400	Blank Input Field.
EIP018	400	Incorrect Input.

Code	HTTP	Message
EPI022	400	Payload is Incorrect.
EIS042	503	Information Source is Not Working.
EUP007	500	Unable To Process. Please Reach Out To Support.
ETP011	400	Incorrect OTP.
EOE794	400	OTP either expired or not generated yet. Kindly regenerate OTP.
EAS517	429	OTP already sent. Please try after 10 min.
EOE082	400	OTP Expired.
ELE077	400	OTP Link Expired.
EAN1390	400	Your Aadhaar has been suspended or cancelled.
ERT788	408	Request Timed Out.

DigiLocker URL And Document Errors

Code	HTTP	Message
EBF017	400	Blank Input Field.
EPI022	400	Payload is Incorrect.
EIP018	400	Incorrect Input.
EIS042	503	Information Source is Not Working.
EUP007	500	Unable To Process. Please Reach Out To Support.
ERT788	408 or 504	Request Timed Out. URL generation maps this to 504; document calls map this to 408.
ENI004	404	No Information Found.

Frontend Implementation Checklist

- Always call KYC APIs with credentials: "include" so the HttpOnly access_token cookie is sent.
- Normalize both top-level success bodies and FastAPI { "detail": ... } error bodies.
- Validate Aadhaar as exactly 12 numeric characters before calling generate-otp.
- Keep reference_id from Aadhaar OTP generation until validate-otp completes.
- Keep kyc_flow_id from DigiLocker URL generation for status checks.
- Plan a backend fix for the kyc_flow_id versus session_id gap before integrating document-list and document-url flows.
- Use documentType, documentFormat, and documentUri exactly as returned by list-documents.
- Use document-precheck on page load to avoid duplicate DigiLocker flows for users who already have stored documents.

Suggested TypeScript Types

```
type ApiEnvelope<T> = {
  message: string;
  data: T | null;
  responseCode?: string | null;
};

type AadhaarOtpResponse = {
  aadhaar_number: string;
  reference_id: string;
};

type DigiLockerGenerated = {
  kyc_flow_id: string;
  digilocker_url: string;
};
```

```
type KycFlowStatus =
  | "CONSENT_PENDING"
  | "IN_PROGRESS"
  | "EXPIRED"
  | "TIMEOUT"
  | "CONSENT_APPROVED"
  | "CONSENT_REJECTED"
  | "ERROR";

type KycDocument = {
  documentType: string;
  documentFormat: string;
  documentUri: string;
  documentUrl?: string;
  [key: string]: unknown;
};
```

Current Contract Caveats

Area	Frontend impact
kyc_flow_id versus session_id	URL generation returns kyc_flow_id, but document APIs require session_id. Frontend cannot complete document retrieval with only documented public responses.
Inconsistent wrappers	Some errors are { "detail": ... }, auth errors are top-level, and success responses vary on responseCode presence.
Missing generate-OTP key	A request body without aadhaar_number can produce 500 due direct dictionary access.
Validate-OTP unmapped response	If ScoreMe returns an unmapped non-SRC001 code, the controller can return no explicit payload.
Duplicate exception helper	The second raise_digilocker_document_list_exception definition overrides the first, so get-document error mapping is used at runtime.
Enum serialization	The controller passes KYC_FLOW_STATUS enum objects into some JSONResponse bodies; plain JSONResponse may not serialize them depending on runtime behavior.
Document URL local update	If document_type does not match a stored documentType, the API still returns the upstream document URL but local update may not happen.

Source Files Reviewed

- routes/kyc_router.py
- controller/kyc_controller/aadhar_kyc_controller.py
- middleware/authorization_middleware.py
- custom_exceptions/scoreme_exceptions.py
- utils/error_codes_utility.py
- config/config.py
- main.py